Company A Orders

Technical Design Document · Retailer ordering app with AI order capture for an FMCG distributor in Jabalpur

GitHub	(add the GitHub URL before submitting)
Live app	Not deployed yet. Run locally with deployment.md.
Brief	FMCG distributor's retailer ordering app with AI order capture (Software Engineering, core build)
Demo logins	see `deployment.md`. OTP is mocked: the code is shown on screen in demo mode.
Date	7 November 2026

1. Summary

The brief asks for one slice to work end to end: **retailer catalogue with prices and schemes, cart and order, AI capture from a text order, and an order desk with Tally export**, for two roles (retailer, order desk), with the scheme engine as the hard part. That slice is built and works on realistic sample data. Around it I added the smaller "must haves" that came almost free once the core was right: repeat order, credit limit with holds, a van loading list, the "no order in 14 days" list, Hindi and English, and an installable app shell.

What works today

- OTP sign-in by mobile number, two roles, per-retailer data isolation.
- Catalogue with the retailer's own price, GST shown on every price, live schemes, stock, low-stock and out-of-stock states, search, repeat last order.
- Cart priced live on the phone by the same engine the server uses: free goods, slab discounts, "add 1 more case, get 1 free" nudges, stock and credit warnings.
- Order placement that re-prices on the server, reserves stock, checks credit and is safe against double taps and races.
- AI capture from a typed or pasted message, with original words, a confidence level, two alternatives, case-or-piece questions, and learning from corrections.
- Order desk: inbox, credit holds, orders, Tally XML/CSV export batches, van loading, quiet

Honest limits

- **The AI reader is Google Gemini on the free tier** (5 requests a minute). It is tested live on the sample messages only (§7.5); with no key, or when Google is busy, the app uses the rule-based reader.
- Photos of lists and Hindi voice notes are read by Gemini and tested with generated files only, not yet with real handwriting or a real voice in a noisy shop (§7.6).
- Salesman offline mode is not built. The app is installable and survives a dropped signal (cart on the phone, offline page), but cannot place orders offline.
- SMS, WhatsApp and Tally are mocked as the brief says: OTP shown on screen, messages pasted by the desk, Tally gets a file.
- Not deployed at time of writing unless a live URL is shown above.

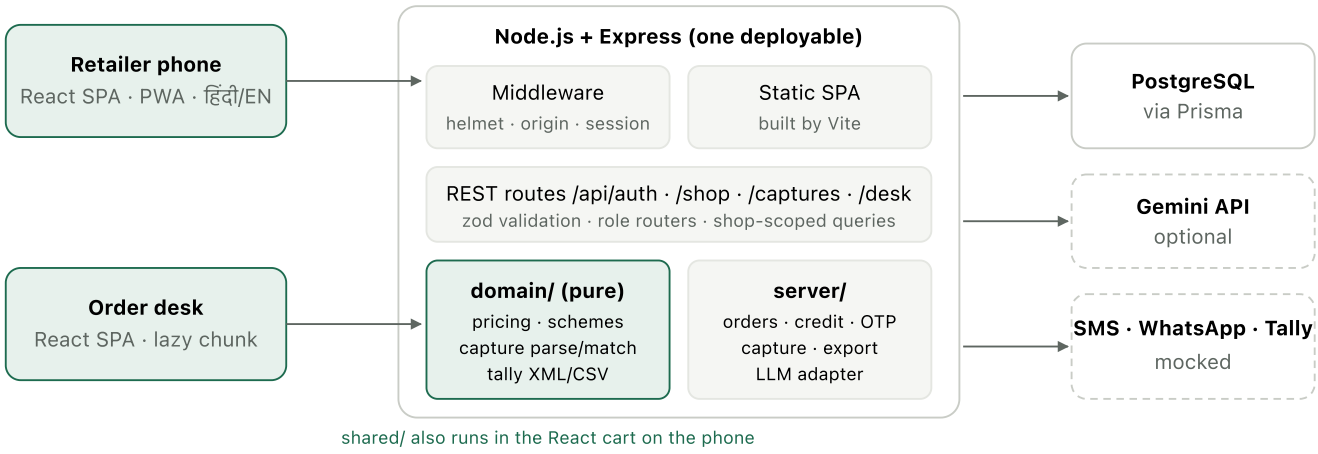
shops.

2. The problem in numbers, and where this slice hits it

The brief puts the cost at ₹52.5 lakh a year: ₹31.5 lakh from 5% of orders having a wrong item or quantity, ₹15.6 lakh of order-entry staff, ₹5.4 lakh of margin lost to stock-outs found after ordering. The design choices map to those three lines:

COST LINE	WHAT CAUSES IT	WHAT THE APP DOES ABOUT IT
Wrong item / qty (₹31.5 L)	Retyping free text; guessing "dabba" means a case; similar SKUs (75 g vs 200 g)	Every captured line keeps the retailer's words next to the SKU; units that are not clearly cases are *asked*; unsure lines must be ticked; corrections become per-shop aliases so the same mistake isn't made twice.
Order entry staff (₹15.6 L)	6 people retype everything into Tally next morning	App orders need no typing. Captured orders need a check, not typing: high-confidence lines are pre-accepted. Export is one click into a Tally import file.
Stock-outs after ordering (₹5.4 L)	Salesman and retailer don't see stock at the counter	Stock is shown per SKU, out-of-stock lines cannot be ordered, the same product's other pack size is offered as a one-tap substitute, and stock is reserved when the order is placed.

3. Architecture



Why this shape

Stack: React + Vite (frontend), Node.js + Express (API), PostgreSQL (database), Prisma (ORM). This is the client's chosen stack. Three folders, one TypeScript codebase, one deployable:

```
shared/   pure TypeScript used by both sides: scheme engine, capture reader, Tally writer, API
types, i18n
server/   Express API: routes → services → Prisma → PostgreSQL
web/      React SPA built by Vite: pages, components, cart state
```

- **One process in production.** Express serves `/api/*` and the built SPA from the same origin. Same origin means the session cookie needs no CORS, and there is one thing to deploy and monitor. In development Vite runs the SPA with hot reload and proxies `/api` to Express.
- **A pure `shared/` layer.** The scheme engine, capture reader and Tally writer have no database, clock or network. That makes them testable with tables of cases, and lets the *same* pricing code run in the React cart (instant feedback, no round trip on 3G) and in Express (the authority when the order is placed). `shared/api.ts` holds every request and response type, so the API and the React app can't drift apart without a compile error.
- **Thin routes, real services.** Routes validate input (zod), check the role and scope, call a service, and map the result to a DTO. Services hold the business rules (orders, credit, capture, OTP) and are tested directly against Postgres.
- **PostgreSQL with row locks.** Stock and credit are shared, contended numbers. Order placement runs in one transaction that locks the retailer row (credit) and the product rows (stock) in a fixed order, then re-prices. A test races two orders for the last cases and checks only one wins.
- **Prisma** for typed queries and migrations; raw SQL only for the two `FOR UPDATE` locks.
- **React Router** with the order desk lazy-loaded, so retailers' phones never download desk screens. No state library: server data comes from a small `useQuery` hook, and the cart is a React context persisted to localStorage.
- **Hosting plan:** one Node service (Render, Railway or a small VM in Mumbai) plus managed Postgres in Mumbai (Neon or Supabase). See `deployment.md`.

Considered and rejected: separate frontend and API hosts (CORS, two deployments, cross-site cookies for no gain at this size); Firebase (weak for transactional stock and credit, and the reporting is relational); a React Native app (the brief asks for an installable web app, and low-end phones have little storage for another app); calling the LLM for everything including SKU matching (opaque confidence, cost per message, and no way to learn per-shop words).

4. Data model

ENTITY	KEY FIELDS	NOTES
User	phone (unique), role RETAILER | DESK, lang, retailerId, active	A retailer user belongs to one shop. Deactivating takes effect on the next request (session is re-checked against the DB).
OtpChallenge	phone, codeHash (HMAC), attempts, expiresAt, consumedAt	Codes are never stored in clear.
Retailer	code (Tally ledger alias), shop, area, route, creditLimitPaise, outstandingPaise, outstandingAsOf	Outstanding is a Tally snapshot; the timestamp matters for credit (A-09).
Product (SKU)	code, name, nameHi, brand, packLabel, unitsPerCase, unitPricePaise, gstRateBp, stockCases, keywords[]	Keywords hold shop slang ("peti", "surf", "chai patti") used by capture.

RetailerPrice	(retailerId, productId) → unitPricePaise	"His price". Missing row = base dealer price.
Scheme	type FREE_GOODS (buyCases, freeCases) | SLAB_DISCOUNT (slabs JSON), validFrom, validTo	Several may exist per SKU; the engine picks one (A-06).
Order / **OrderLine**	number, status CONFIRMED | CREDIT_HOLD | EXPORTED | REJECTED, source APP | CAPTURE, totals; line: cases, freeCases, prices, discount, GST, schemeLabel	Lines snapshot prices so later price changes never rewrite history. `idempotencyKey` is unique.
Capture / **CaptureLine**	rawText, channel, parser, status; line: rawText, phrase, qty, unit CASE | PIECE | UNKNOWN, productId, confidence, matchedBy, alternatives	The draft a person reviews. Nothing becomes an order until confirmed.
RetailerAlias	(retailerId, phrase) → productId	Learned from corrections, per shop.
ExportBatch, **AuditLog**	batch of orders sent to Tally; who did what, when	Batches can be downloaded again. The order page shows the audit trail.

Money is integer paise everywhere; quantities are whole cases. Floating point never touches a rupee.

---

5. Main flows

5.1 Retailer orders in the app

1. Sign in with mobile and OTP. Session cookie: httpOnly, signed, 30 days for retailers (they order between customers and should not re-login daily), 12 hours for the desk.
2. Catalogue shows his price per case, "+ GST %", price per pack, live scheme, and stock. "You order often" and "Repeat last order" sit on top; repeat skips anything now out of stock and says so.
3. The cart lives on the phone (localStorage), so tapping + is instant and nothing is lost if the signal drops. It is priced by the shared engine: free cases, discounts, nudges, GST breakdown, and a warning if the total goes over available credit.
4. `POST /api/shop/orders` sends only product ids, case counts, an idempotency key and the total the person saw. The server re-prices inside a locked transaction. If its total differs (price or stock changed), it refuses and the cart refreshes with a clear message instead of silently charging a different amount.
5. Within credit → `CONFIRMED`; over credit → `CREDIT_HOLD` with the exact shortfall. Stock (paid + free) is reserved either way.

5.2 Message to order (the AI part)

1. The desk pastes a WhatsApp text, adds a photo of a list or a voice note, and picks the shop (WhatsApp is mocked). Or the retailer types the order, photographs their list, or records a voice note in the app.
2. The reader splits lines, extracts quantity, unit and pack size, and matches each line to a SKU (§7). A **Capture** draft is saved.

3. The review screen shows, per line: their words, the matched SKU with stock, a confidence label and %, why it matched, two alternatives, a full product picker, and the case-or-piece question where needed. A live preview prices the draft with the scheme engine, so the desk sees free goods and stock problems before confirming.
4. Confirm is blocked until every line is answered. `POST /api/captures/:id/confirm` enforces the same rules on the server (the UI cannot skip them), creates the order through the same `placeOrder` service, marks the capture done and saves corrections as aliases, all in one transaction.

5.3 Order desk

Inbox (oldest message first, credit holds, counts) → review → order page (release hold, reject with a reason the shop sees; rejecting returns stock) → Tally export (select confirmed orders, create a batch, download XML or CSV; a status-guarded update prevents two people exporting the same order) → van loading list by route (paid + free cases) → quiet shops (no order in 14 days, quietest first, tap to call).

---

6. The scheme engine (the hard part)

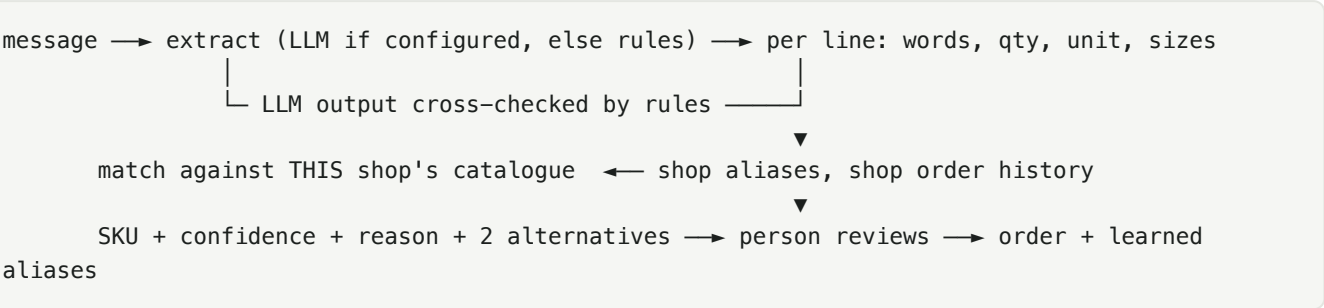
`shared/pricing.ts` is pure and shared by the React cart and the Express API. Rules, each covered by tests:

RULE	BEHAVIOUR
Case price	pack price × packs per case. Biscuit 75 g: ₹8.10 × 96 = ₹777.60; detergent 1 kg: ₹101.50 × 20 = ₹2,030.00; tea 250 g: ₹118 × 40 = ₹4,720.00 (A-01).
Free goods	floor(cases ÷ buy) × free, repeating: 10 → 1 free, 20 → 2. No charge and no GST on free cases (A-07, verify).
Slab discount	Highest slab reached applies to the whole line; slabs may be unsorted. GST is charged on the discounted value.
One scheme per line	Never stacked. If two are live, the one worth more to the retailer *at that quantity* wins, ties by id so pricing is deterministic (A-06).
Validity	Live when validFrom ≤ now ≤ validTo; expired and future schemes are ignored; malformed ones (buy 0) are ignored rather than dividing by zero.
Stock	Out of stock or above stock blocks the line. Free goods come from the same stock: if not enough is left, give what is left and flag it without blocking.
Nudges	"Add N more cases, get 1 free" / "for 3% off" when within half the threshold and stock can cover it (A-08).
Rounding	Integer paise, half up, per line (A-05). Lines that are blocked are priced for display but excluded from totals.
Duplicates	Two cart lines of the same SKU are merged before pricing, so a threshold can't be split or doubled.

Worked example from the brief. Detergent 1 kg, 5 cases: gross ₹10,150.00, 3% = ₹304.50, taxable ₹9,845.50, GST 18% = ₹1,772.19, total **₹11,617.69**. At 2 cases (the brief's sample order) there is no discount and the cart says "Add 3 more cases for 3% off". Biscuit at 5 cases says "Add 5 more cases, get 1 free". Tea 250 g has 0 stock and is blocked, with tea 500 g offered as a substitute.

7. AI order capture

7.1 Pipeline



Extraction splits on commas, new lines, "aur/and", numbering and danda; reads digits, Devanagari digits and number words (ek, do, teen, पांच…); separates pack sizes ("1 kilo", "250 wala", "75") from quantities; and reads units: *peti, case, carton* = case; *pkt, pc, nag* = piece; *dabba, box* or nothing = **unknown**. "do" is two before a unit and "give" at the end ("bhej do").

Matching scores every SKU: word coverage against name, Hindi name, brand and slang keywords (exact, phonetic for Hinglish spellings such as biskut / biscuit, prefix and edit distance), a bonus for intact phrases ("washing powder"), +20 for a matching pack size and −25 for a conflicting one, and up to +8 for SKUs this shop orders often.

7.2 When it is unsure or wrong

SITUATION	WHAT HAPPENS
Two SKUs score close ("biscuit" alone)	Confidence is reduced by 1.5 × the missing margin below 15 points, so a near tie can't show as "Sure". The shop's history breaks the tie and the screen says "Picked from this shop's past orders".
Nothing matches well (score < 35)	No guess. The line shows "Choose the product" with the best alternatives and the full list.
Unit unclear ("10 dabba", "biscuit 5")	Unit stays UNKNOWN and the person must answer Cases or Pieces. Never guessed (brief).
Pieces ("ketchup 24 pkt")	We sell cases (A-04). 24 pieces of a 24-pack case is shown as "= 1 case"; a non-whole number asks for cases.
Quantity missing or a fraction ("aadha peti")	Left empty; the person enters cases.
Medium / low confidence	The person must tick "I've checked this line". High-confidence lines are pre-accepted, so the desk only reads the doubtful ones.
Matched SKU is out of stock	One-tap substitute with the same product's other pack. Substitutes are **not** learned: tea 500 g is not what "chai patti 250" means.
Wrong SKU picked by the reader	The person picks an alternative or any product. On confirm, the shop's words become an alias for that SKU, for that shop only. Next time the same words match at 97% with "Learned from an earlier correction".
LLM fails, times out or returns bad output	Falls back to the rule reader; the review screen says so. The desk still gets a draft.

7.3 Measured on a labelled evaluation set

To judge matching on a realistic catalogue rather than an easy one, 22 deliberate look-alikes were added (43 SKUs): more pack sizes of the same product, milk vs glucose vs cream biscuits, tea leaf vs tea dust 1 kg, rock vs iodised salt, liquid detergent vs dishwash liquid, two neem soaps. `shared/capture/eval` holds 30 messages typed the way shops type (no commas, Hinglish, Devanagari, typos, brand slang, missing units, a product we don't sell), with 49 lines labelled the way a careful desk clerk would decide. `npm run eval` scores it; a test fails the build if the rule reader drops below the floor.

READER	RIGHT SKU	QTY / UNIT	WRONG BUT "SURE"	AUTO-ACCEPTED	AMBIGUOUS HANDLED
Rules, first run	83.7%	93.9% / 93.9%	0	79.6%	7/7
Rules, after fixes, no shop history	91.8%	100% / 100%	0	79.6%	7/7
Rules, after fixes, with shop history	95.9%	100% / 100%	0	91.8%	7/7
Gemini + guardrails, with history	**98%**	100% / 100%	0	91.8%	7/7

The first run's misses had general causes, fixed as rules rather than per-message patches: messages without commas are cut at each "number + peti"; a longer, different word is no longer a prefix match ("rocket" had matched rock salt); "4 pack wala" is read as a description, not the order; Devanagari size words (बड़ा) count like "bada". What the rules still miss is brand slang ("maggi" for noodles, "vim" for a dishwash bar), shown as *low* so the desk is asked. Gemini fixed exactly those two; its only miss was suggesting noodles for "rocket", which the guardrails labelled an AI suggestion at *medium*. "Dangerous" errors (a wrong product shown as "Sure", which a busy desk would wave through) stayed at zero in every run. The set is small and hand-written; the next step is to label 200 real WhatsApp orders from the client.

7.4 Why rules first, LLM optional

Rules are free, instant, offline-capable, deterministic and explainable, and the vocabulary is narrow (43 SKUs, a few dozen slang words). Their weakness is open-ended phrasing. An LLM is good at exactly that, but its confidence is not calibrated and it can't learn a shop's words. So the LLM is used only to *extract and normalise* lines (structured output against a schema), and the deterministic matcher still decides the SKU and the confidence.

7.5 The LLM path (Google Gemini) and its guardrails

`server/src/services/llm.ts` uses Google's official `@google/genai` SDK with a Google AI Studio key. It asks `gemini-3.5-flash-lite` for JSON constrained to a response schema, at temperature 0 and low thinking, with the catalogue terms in the system instruction, then validates the reply again with zod. A blocked prompt, a safety or length stop, non-JSON or wrong-shaped output, a quota error or a timeout (15 s) all raise, and capture falls back to the rule reader; a transient 503 gets one quick retry first. Enabled with `LLM_PROVIDER=gemini` and `GEMINI_API_KEY`; `LLM_MODEL` pins another model.

The model suggests; the retailer's words decide. Running it against the samples showed three ways a model quietly overstates what the shop wrote, and each now has a rule:

WHAT GEMINI DID	RISK	RULE NOW
Called "biscuit 5" a *glucose* biscuit	Ambiguous line jumped from "check, 63%" to "sure, 100%", so the desk would not be asked	Gemini's words are kept apart (`aiWords`). Confidence comes from the shop's own words; agreement adds at most 15 points; an AI-only pick is labelled "AI suggestion" and capped at medium. Aliases are keyed on the shop's words only.
Filled in "150 g" for "lal wala sabun" (no size written)	An invented pack size boosted the match to 78%	A model pack size is used only if its number appears in the shop's words.
Kept "4)" list numbering in its copy of the line	The 4 was read as a pack size	The line parser strips list numbering itself.
Said "CASE" for "10 dabba"	A guessed unit	The unit always comes from the shop's words; "dabba" stays a question.

Measured (free tier, 4 sample messages, 14 lines): same SKU, quantity and unit as the rule reader on every line; clear lines rise from 85% to 95%; "biscuit 5" stays *medium 73%* and "lal wala sabun" *low 36%*, so both still need a person; 1.7–4.6 s per message. Larger "flash" models took 12–15 s or more and were often overloaded (503), so the lite model is the default. `npm run ai:check` repeats this comparison.

> Free-tier limits: 5 requests per minute per model, occasional 503s, and Google may use free-tier prompts to improve its products. Fine for a demo; the pilot needs a paid key (about 600 messages a day), and the fallback to rules means a quota hit never stops the desk. On the evaluation set Gemini's gain is real but narrow (brand slang); its value should be confirmed on the client's real messages before paying for it at scale.

7.6 Photos of lists and Hindi voice notes

Photos and voice notes need an AI that reads images and audio, so they use Gemini; without a key the buttons explain that and point to typing. The flow reuses everything above:

1. **On the phone:** a photo is scaled to 1600 px JPEG (a 4 MB camera photo becomes about 250 kB on 3G); a voice note is recorded in the app (up to 90 s) and converted to 16 kHz mono WAV, a format every browser can produce and Gemini always reads. The desk can also upload a forwarded WhatsApp voice note (.ogg/.opus, .m4a, .mp3).
2. **Upload:** the raw file goes to `POST /api/captures/media`. The server checks the type against an allow-list, caps it at 8 MB, and sniffs the first bytes so a file can't pretend to be a photo.
3. **Transcribe, don't interpret:** Gemini writes down what is on the list or in the note, one item per line, in the original script (spoken Hindi as Roman Hinglish), with numbers as digits. It may not translate, correct or add, and writes `[unclear]` instead of guessing. Something that isn't an order (a selfie, silence) is refused.
4. **Same reader, stricter rules:** the transcript goes through the same reader, matcher and guardrails as a typed message. Because a transcript can mishear "do" (2) as "dus" (10), every line from a photo or voice note is capped at *check* (79%), so a person ticks each one, and an `[unclear]` line is *low*. The review screen shows the photo, or plays the voice note, above the transcript.

5. **No learning from transcripts:** corrections on a photo or voice capture are not saved as the shop's words, because those words came from the AI, not the shop.

Measured live (free tier, files generated for the test: a photo of a five-item list in a handwriting font, Roman and Devanagari mixed; a Hindi voice note spoken by macOS's Hindi voice): the photo was transcribed exactly in 2.8–4.9 s end to end, all five SKUs matched, and "24 pkt" correctly stayed pieces. The voice note took 3.3–3.9 s; on one run Gemini heard "सर्फ" (surf) correctly, on another it heard "salt". That is the case the 79% cap, the playable original and the no-learning rule exist for. Not yet tested: real handwriting on a real phone camera, a real person's voice with shop noise, and in-app recording on a phone (the test browser has no microphone; its "microphone blocked" message was checked).

---

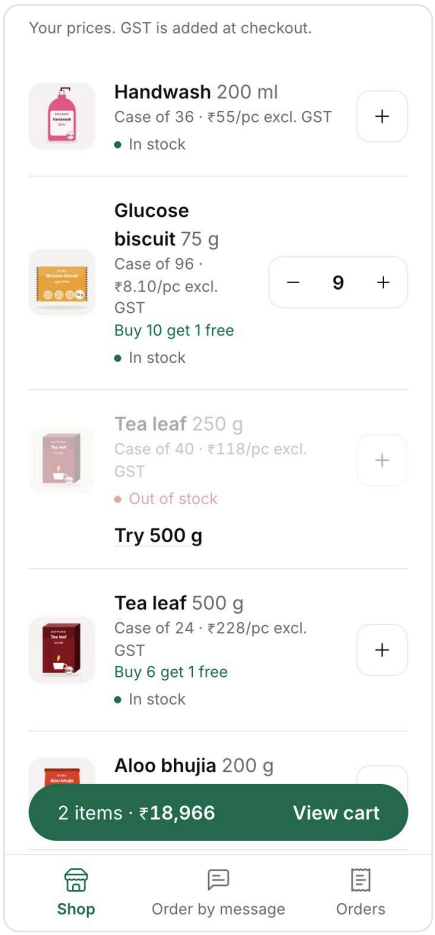
8. Security and privacy

- **Access control on the server, every request.** A middleware reads the signed session cookie and re-loads the user from the database, so a deactivated user is locked out at once. Each router checks the role (`/api/shop` retailer, `/api/desk` desk). Every retailer query is scoped by the shop id from the session, never from the URL: another shop's order id returns 404, so ids don't leak. The React route guards are only for navigation; the API is the gate. Covered by API tests.
- **OTP.** 6 random digits, HMAC-hashed, 5-minute expiry, 5 attempts counted atomically before comparing (parallel guesses can't exceed the limit), constant-time compare, 3 codes per number per 10 minutes, 30 s between sends, plus a per-IP limit. The response is identical for registered and unknown numbers, so the form can't reveal who is a customer.
- **Sessions.** HS256-signed token in an httpOnly, SameSite=Lax cookie (Secure in production); 30 days for retailers, 12 hours for the desk.
- **CSRF.** State-changing API calls must be JSON (blocks plain cross-site form posts) and come from the app's own origin or an allowed one. SameSite=Lax on top.
- **Validation.** zod on every request body, query and environment variable; case counts are whole numbers 1–999; message length capped; JSON bodies capped at 64 kB; product ids checked to exist.
- **Uploads.** Photos and voice notes only (allow-list), 8 MB at most, and the file's first bytes must match its declared type. Files are stored in the database, served only to that shop and the desk, never executed, and sent to Gemini only when the AI reader is on.
- **Headers.** helmet with a strict Content-Security-Policy (scripts only from self, no frames), nosniff, referrer policy; `x-powered-by` off.
- **Output safety.** XML is escaped; CSV cells starting with `= + - @` are neutralised against spreadsheet formula injection. Errors return a generic message; details go to the server log only.
- **Secrets.** None in the repository: `.env` is git-ignored, `.env.example` lists names only. No real personal data: all shops, people and phone numbers (90000 000xx) are invented.

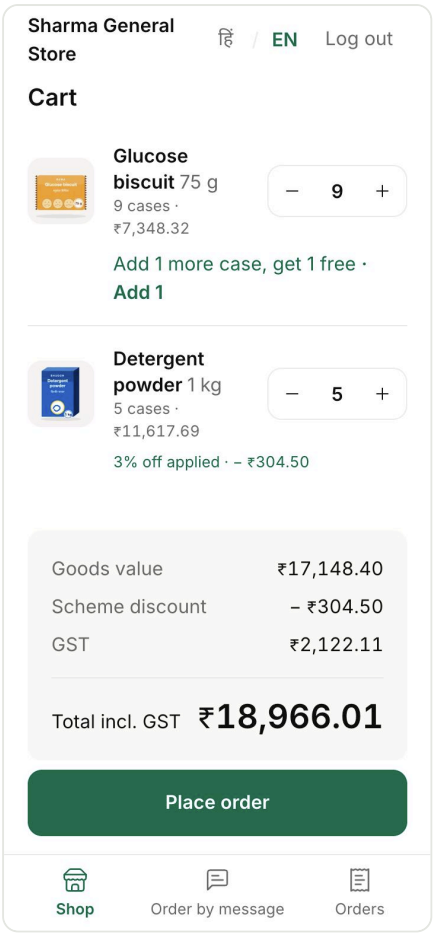
9. UI and design decisions

Kirana owners order between customers, on low-end Android phones, in Hindi or English. The app is quiet on purpose: white space instead of boxes, hierarchy from size and weight instead of colour, and one obvious thing to do on each screen.

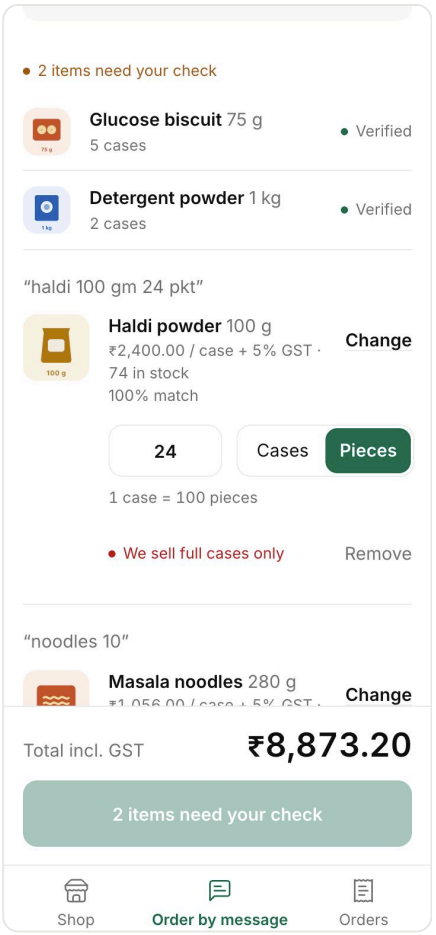
- **One primary action per screen:** *Reorder last order*, *Place order*, *Confirm order*. Everything else is plain text.
- **One accent colour** (deep green #0F6B4F), only for the primary button, the active tab and the cart bar; scheme lines use it as plain text because a scheme is money for the shop.
- **Status is a small dot plus words** (green verified, amber check, red blocked), never a filled badge and never colour alone.
- **Three type sizes** (16 body, the floor for anything important; 20 titles; 28 totals) plus 13 for captions; weights 400 and 600; tabular numbers for prices; secondary text #6B6B6B (5.3:1, WCAG AA). Inter and Noto Sans Devanagari, 98 kB in total with Inter preloaded, Hindi at line-height 1.7.
- **Hindi first-class:** a हिं / EN switch always in the header; no fixed widths and no truncation, so 30%-longer Hindi wraps instead of being cut.
- **No shadows, gradients or emojis;** hairline borders where depth is needed; outline icons at 1.5 px, sparingly. Touch targets are at least 44 px.
- **Show less, reveal on tap:** verified review lines fold into one row; the original message is two lines until expanded; a photo or voice note opens on request.



Home. Greeting with credit left, one reorder button, search, then one product per row. The shop's usual products sort to the top. The cart bar appears only when the cart has items.

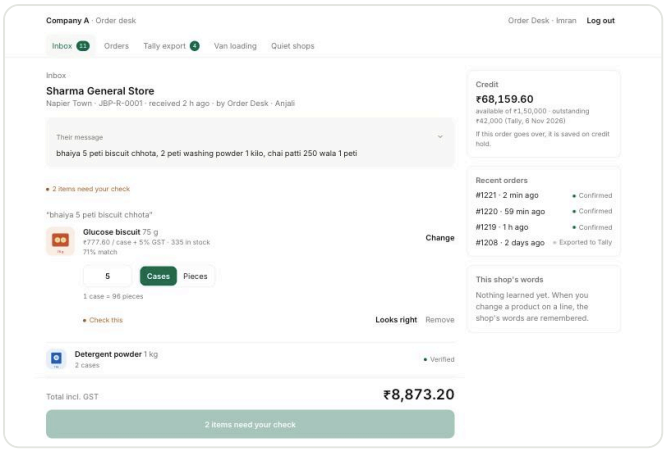


Cart. A plain list, a quiet summary and the total at 28 px. Scheme hints are one tap ("Add 1 more case, get 1 free · Add 1").

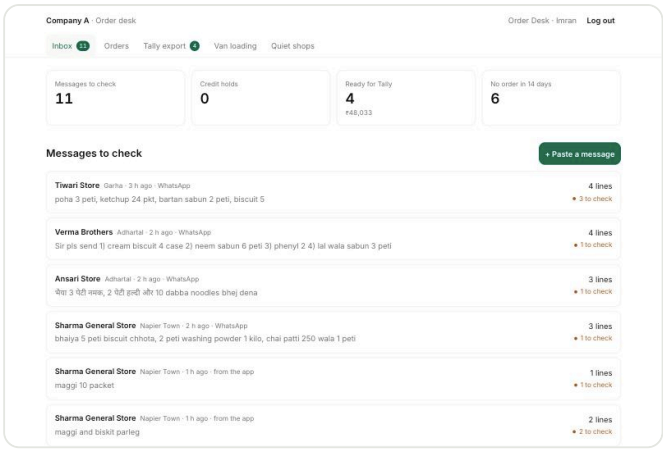


Order review. Verified lines fold away; open lines show the shop's words, product, price, stock and match %, one quantity box and a Cases/Pieces switch with "1 case = 100 pieces", and one short reason. Confirm says how many items still need checking.

The order review screen follows ten rules: no repeated unit prompts or long AI explanations; a bottom sheet with suggestions and search-as-you-type instead of a 43-item dropdown; the shop's words, product, match %, the shop's own price and stock on every line; one quantity box and one Cases/Pieces switch, never both units at once; "peti/case" selects Cases and "packet/piece" selects Pieces; anything else asks instead of guessing; the pack conversion is always shown; a missing quantity is never filled in; open lines and verified lines look clearly different; and *Confirm order* stays disabled until every product, quantity and unit is settled. The backend contract did not change.



Desk review. The same review component on a desktop, with credit, recent orders and learned words alongside. A doubtful line (71% match) asks for "Looks right".



Desk inbox. Four counts that drive the day, then messages oldest first with "N to check", so the desk knows the effort before opening one.



Product images. One rendered pack shot per SKU, generated from the catalogue by `scripts/render-pack-images.ts`. Look-alikes stay distinguishable: three glucose biscuit sizes, tea leaf vs tea dust, rock vs iodised salt, liquid detergent vs dishwash liquid.

Smaller decisions: every row says "excl. GST" with the per-piece price, and the cart itemises GST, because dealer invoices are ex-GST and rates differ by product (0, 5, 18%). The login resends an OTP only after a 30-second countdown that matches the server's limit. The desk decides whether a correction becomes the shop's saved word ("Save 'kala' as this shop's word", ticked by default). Unit questions name the specific problem ("Cases or pieces?") rather than a generic "Check this". The bottom navigation has three places (Shop, Order by message, Orders) and no cart tab, because the cart

bar already carries it. Product images are rendered pack shots (the brands are fictional, so there are no real photos to use): one per SKU, drawn from the catalogue with the right packaging, colour, names and pack size, 2.4 kB each, lazy-loaded. The order desk got the same tokens but no layout redesign. The full rationale is in `docs/design-rationale.md`.

---

10. Performance on slow 3G

- Retailer bundle: **115 kB** JavaScript + 5.9 kB CSS, gzipped. Most of it is React, React DOM and React Router (icons add about 4 kB); the app's own code is small. Order desk screens are separate chunks (1–2 kB each) that retailers never download.
- Fonts: 98 kB in total, inside a 100 kB budget. Inter 400 and 600 (24 kB each, 400 preloaded) and Noto Sans Devanagari 400 (50 kB), self-hosted woff2 with `font-display: swap`; the Devanagari file is downloaded only when a screen shows Hindi. Product images: 192 px WebP pack shots of about 2.4 kB each (103 kB for all 43), lazy-loaded so the first screen fetches about 16 (about 38 kB), cached for a year and kept offline by the service worker. Static assets carry content hashes and a one-year cache; the service worker keeps them for repeat visits and shows an offline page instead of the browser's error.
- The cart is local, so adding cases costs no network at all; the catalogue API response is one request of about 2.3 kB gzipped (9.8 kB raw). All responses are gzip-compressed by the server.
- Skeletons shaped like the content appear immediately, so the screen is never blank while data loads.
- **Honest estimate:** on Chrome's "Slow 3G" profile, a *first ever* visit is about 3–3.5 s to a usable catalogue (a single-page app shows content after its JavaScript loads); repeat visits, which are almost all visits for a shop ordering every few days, are about 1–1.5 s from the cache. That misses the brief's "under 3 seconds" on the very first visit. The cheapest fixes, in order: swap React DOM for Preact (about –35 kB), and prerender the catalogue shell. Not yet measured on a real low-end phone; that is a pilot-week task.

---

11. Tests

151 tests (Vitest + supertest), run in CI against Postgres together with lint, typecheck and a production build:

- **Scheme engine, 44:** the brief's three SKUs, every free-goods threshold and nudge, slab boundaries and unsorted slabs, competing schemes, validity edges, stock caps, invalid quantities, rounding, merging duplicates, credit maths.
- **Capture reader, 47:** including the evaluation-set quality floor (0 dangerous errors, 100% qty and unit, ≥95% SKU); the brief's message end to end, units asked not guessed (Hinglish and Devanagari), sizes vs quantities, list numbering, splitting, misspellings, no-guess on unknown words, history tie-break, aliases, two alternatives always offered; AI words can lift confidence by at most 15 points, an AI-only pick is capped at medium, and a shop alias beats the AI.
- **Tally export, 7:** XML escaping, India date, actual vs billed quantity, debit = credits balance, CGST/SGST split, CSV quoting and formula neutralising.
- **Photo/voice uploads, 10:** type sniffing (an HTML page sent as a JPEG is refused), size and allow-list; a photo capture stores the file, caps every line at *check*, marks `[unclear]` lines low, and learns

no aliases from corrections.

- **Services against Postgres, 10:** server re-pricing and stock reservation, credit hold, two small orders can't both pass, out-of-stock refused, changed total refused, idempotency, **two concurrent orders for the last cases**, reject returns stock; capture can't be confirmed with unanswered units, by another shop, or twice; corrections are learned.
- **Gemini adapter (mocked), 18:** a good answer becomes lines; not-JSON, wrong shape, cut off, safety stop, blocked prompt, network error and timeout all fall back; one retry on 503 but none on a quota error; the model never decides the unit; a quantity disagreement empties the field; an invented pack size is ignored.
- **HTTP API, 15:** including uploads (503 when the AI reader is off, 400 for a disguised file, 415 for other types, another shop can't fetch a shop's photo); no session → 401; retailer → desk endpoints 403; another shop's order → 404; deactivated user and tampered token locked out; desk can read any order; cross-origin write → 403; form-encoded write → 415; validation messages; bad quantities rejected; unknown phone gets the same answer as a known one.

---

12. Assumptions

Only the brief was used. Where it is silent, I decided and labelled it here. Items marked **verify** need the client.

#	ASSUMPTION
A-01	The sample "Dealer Price" is per pack; case price = pack price × packs per case (₹8.10 is a sensible price for a 75 g biscuit, not a case of 96).
A-02	Dealer prices exclude GST. Every price says "+ GST %"; the cart shows the GST-inclusive total.
A-03	GST rates are placeholders: packaged food 5%, salt 0%, home care 18%. **Verify** with the client's accountant; not tax advice.
A-04	Retailers order whole cases. Pieces are converted to cases where exact, otherwise the reviewer enters cases. Loose-piece selling is out of scope.
A-05	Money in integer paise; discount and GST rounded half up per line.
A-06	One scheme per SKU line, never stacked; if several are live, the one worth most to the retailer at that quantity.
A-07	Free goods use the same stock, carry no charge and no GST. Whether GST applies to free supplies is for the accountant: **verify**.
A-08	Nudges appear within half of the next threshold (rounded up) and only if stock can cover it.
A-09	Available credit = limit − outstanding from the last Tally sync − orders accepted since that sync. Held orders don't count until released.
A-10	An order over credit is accepted on hold, not refused, and the retailer is warned before placing it. The desk releases or rejects.

A-11	Stock is reserved when an order is placed (including holds) and returned on rejection. Stock figures would come from Tally (mocked).
A-12	Retailer screens are Hindi and English; the order desk works in English. Hindi copy needs a native review.
A-13	Corrections by whoever confirms (desk, or a retailer on their own order) become aliases scoped to that shop. Substitutions for stock are not learned.
A-14	Orders go to Tally as *Sales Order* vouchers; invoicing and e-invoicing stay in Tally. Ledger names ("Sales GST 18%", "Output CGST 9%", party "Shop (code)") are placeholders that must match the client's masters; intra-state (MP), so CGST + SGST. **Verify**.
A-15	WhatsApp is mocked: the desk pastes a message and picks the shop (in production the sender's number identifies it).
A-16	Van loading list = accepted orders from the last 24 hours, grouped by the shop's route.
A-17	All brands, shops, people and numbers in the seed are invented. Schemes in the seed run from 36 days before to 60 days after seeding so the demo never expires.
A-18	Sessions last 30 days for retailers and 12 hours for the desk.

13. What I cut, and what I would do next

Cut, on purpose

- **Salesman mode and offline ordering.** Needs a local queue (IndexedDB), conflict rules for stock and credit, and a sync screen. Too risky to do badly.
- **Real SMS, WhatsApp and Tally integration** (mocked, per the brief).
- **Admin screens** for products, prices, schemes and users (seeded instead).
- Payments, delivery tracking, e-invoicing (out of scope).

Next, in order

1. Turn on Gemini with a key; build an evaluation set from 200 real WhatsApp orders; tune thresholds on it and track accuracy weekly.
2. WhatsApp Business API inbound webhook, so messages arrive in the inbox by themselves, matched to the shop by number.
3. Test photo and voice capture with 50 real lists and voice notes from shops; measure mishearing of numbers and tune the prompt.
4. Salesman mode: offline catalogue and order queue, sync with server-side re-pricing (the engine already supports it).
5. Scheme and price admin for the office, with a preview using the same engine.
6. Tally sync for stock and outstanding (ODBC/XML), replacing the mocked snapshots.
7. Measure on a real low-end phone on 3G; Preact and a prerendered shell if the first visit is over 3 s; error monitoring.